

QUIZON Rosselle

TP Docker / Volumes / Réseaux & Kubernetes (Minikube)

Objectifs du TP

Ce TP avait pour but de découvrir Docker et d'introduire Kubernetes :

- manipuler images et conteneurs (docker run, docker ps, docker images) ;
- comprendre la différence entre conteneur et image (export/import) ;
- gérer la persistance des données avec les volumes Docker ;
- lancer un serveur MySQL dans Docker avec un volume ;
- découvrir les réseaux Docker (bridge, host, none, réseau personnalisé) ;
- construire des images avec des Dockerfile (Alpine, Java, Flask) ;
- enfin, utiliser Minikube + kubectl pour déployer Nginx dans Kubernetes.

Premiers conteneurs: hello-world et Alpine

On commence par vérifier que Docker fonctionne :

```
docker run hello-world
```

Cette commande télécharge l'image hello-world et lance un conteneur qui affiche un message de test.

On liste ensuite les images et les conteneurs avec :

```
docker images
```

```
docker ps      # conteneurs en cours d'exécution
```

```
docker ps -a   # tous les conteneurs, même stoppés
```

On lance ensuite une image minimale Linux (Alpine) en mode interactif :

```
docker run -it alpine:latest /bin/sh
```

Puis avec un nom explicite :

```
docker run --name monconteneur -it alpine:latest /bin/sh
```

Pendant que ce conteneur tourne, docker ps affiche par exemple :

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
999d1595de59	alpine:latest	"/bin/sh"	9 seconds ago	Up 8 seconds	monconteneur

Cela montre qu'un conteneur est une instance en cours d'exécution d'une image (ici alpine:latest).

Modifier un conteneur et en faire une nouvelle image (export / import)

Dans le conteneur Alpine, on pouvait par exemple créer un fichier :

```
echo "Bonjour depuis le conteneur" > /hello.txt  
ls
```

Puis, sans faire exit, on est retourné dans un autre terminal pour « figer » l'état du conteneur.

Exporter le conteneur

Depuis l'hôte (toujours en root) :

```
docker export 999d1595de59 -o my_container.tar  
ls -ilah
```

On obtient un fichier my_container.tar d'environ 8,3 Mo, qui contient le système de fichiers du conteneur.

Importer comme nouvelle image

On transforme ensuite cet export en une nouvelle image Docker :

```
docker import my_container.tar my_new_image:v1  
docker images
```

On voit alors :

```
alpine:latest      ...  
hello-world:latest ...  
my_new_image:v1    ...
```

On relance un conteneur à partir de cette nouvelle image :

```
docker run --name cmodifie -it my_new_image:v1 /bin/sh
```

Une image peut donc être créée à partir de l'état d'un conteneur (export/import), en plus des Dockerfile.

Volumes Docker : persistance des données

On commence par créer et supprimer un volume :

```
docker volume create monvolumedemo  
docker volume ls  
docker volume inspect monvolumedemo  
docker volume rm monvolumedemo
```

On voit que le volume correspond à un répertoire sur l'hôte, typiquement :

```
/var/lib/docker/volumes/monvolumedemo/_data
```

Volume + conteneur Alpine (données persistantes)

On crée un volume et on le monte dans /data à l'intérieur du conteneur :

```
docker volume create monv
docker run -it -v monv:/data alpine sh
```

Dans le conteneur :

```
ls      # on voit le dossier data
cd /data
echo "Bonjour écrit à l'intérieur du conteneur" > /data/hello.txt
ls
more hello.txt
```

On quitte et on supprime le conteneur (docker rm ...), puis on en relance un nouveau sur le même volume :

```
docker run --name conteneuravecvolume -it -v monv:/data alpine sh
cd /data
ls
more hello.txt
```

Le fichier hello.txt est toujours présent donc la donnée est stockée dans le **volume**, pas dans la durée de vie du conteneur.

Bind mount vers le répertoire courant

On a aussi testé un bind mount :

```
docker run -it -v ./app alpine sh
```

Ici, . (répertoire courant sur l'hôte) est monté dans /app dans le conteneur.
Tout fichier créé dans /app se retrouve directement sur la machine hôte.

MySQL dans Docker + volume mysql_data

On crée un volume dédié :

```
docker volume create mysql_data
```

Puis on lance MySQL dans Docker :

```
docker run -d --name mysql \  
-e MYSQL_ROOT_PASSWORD=pass \  
-v mysql_data:/var/lib/mysql \  
mysql:8
```

Lors de ce lancement, Docker a téléchargé l'image MySQL 8 car elle n'était pas disponible localement.

Connexion au serveur MySQL dans le conteneur

Nous nous sommes connectés à MySQL via :

```
docker exec -it mysql mysql -uroot -ppass
```

Création d'une base de données et d'une table

Dans le shell MySQL, nous avons créé une base de données :

```
CREATE DATABASE demo;  
USE demo;
```

Puis une table :

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100)  
);
```

Insertion de données :

```
INSERT INTO users (name, email) VALUES  
( 'Alice', 'alice@example.com'),  
( 'Bob', 'bob@example.com'),  
( 'Charlie', 'charlie@example.com');
```

Vérification :

```
SELECT * FROM users;
```

Résultat obtenu :

id	name	email
1	Alice	alice@example.com
2	Bob	bob@example.com
3	Charlie	charlie@example.com

4. Persistance des données (test d'arrêt/redémarrage)

Nous avons arrêté le conteneur :

```
docker stop mysql
```

Puis relancé :

```
docker start mysql
```

Connexion à nouveau :

```
docker exec -it mysql mysql -uroot -ppass
USE demo;
SELECT * FROM users;
```

On constate que les données sont toujours présentes.

Cela confirme que le volume Docker mysql_data stocke réellement les données MySQL, indépendamment du conteneur.

Recréation d'un conteneur avec un mount bind

Nous avons ensuite relancé un second conteneur, utilisant toujours le même volume :

```
docker run -d --name mysql1 \
  --mount source=mysql_data,target=/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=pass \
  mysql:8
```

Connexion :

```
docker exec -it mysql1 mysql -uroot -ppass
USE demo;
SELECT * FROM users;
```

On remarque que les données sont toujours présentes dans mysql1, car les deux conteneurs utilisent le même volume Docker (mysql_data).

Cela montre bien que les données ne sont pas stockées dans le conteneur, mais dans le volume. Un conteneur peut donc être supprimé/recréé sans perdre la base de données.

Nous avons ensuite stoppé le conteneur : `docker stop mysql1`

Réseaux Docker

On liste d'abord les réseaux :

```
docker network ls
```

Résultat :

- bridge : réseau par défaut des conteneurs
- host : partage le réseau de la machine (uniquement Linux)
- none : conteneurs sans réseau

Bridge (par défaut)

```
docker run -d --name web nginx
docker network inspect bridge
```

Le conteneur est attaché au réseau bridge avec une IP interne.

Host

```
docker run --network host nginx
```

Nginx écoute directement sur le port 80 de la machine.

None

```
docker run --network none -it alpine sh
```

Le conteneur n'a aucune connectivité réseau (isolation maximale).

Réseau personnalisé

```
docker network create monreseau
docker run -d --name site --network monreseau nginx
docker run -d --name api --network monreseau alpine sleep 1000
docker exec -it site ping api
```

Les conteneurs peuvent se ping par leurs noms grâce au DNS interne.

On peut connecter/déconnecter dynamiquement :

```
docker network connect monreseau site  
docker network disconnect monreseau site
```

Publication d'un port

```
docker run -d -p 8080:80 nginx  
docker ps
```

On voit par exemple :

```
0.0.0.0:8080->80/tcp nice_neumann
```

L'application Nginx est accessible sur <http://localhost:8080>.

Dockerfile Alpine minimal

```
FROM alpine:latest  
CMD echo "Bonjour Polytech Nancy"
```

Build & run :

```
docker build -t mon-app .  
docker run mon-app
```

Résultat :
Bonjour Polytech Nancy

Dockerfile Java

Dans le dossier 2decembre, on a le Dockerfile

```
FROM eclipse-temurin:21-jdk AS build  
WORKDIR /app  
COPY Hello.java .  
RUN javac Hello.java  
FROM eclipse-temurin:21-jre  
WORKDIR /app  
COPY --from=build /app/Hello.class .  
CMD ["java", "Hello"]
```


Avec le fichier Hello.java :

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Bonjour Polytech Nancy");  
    }  
}
```

On a donc écrit un premier Dockerfile, compilé un programme Java dans un conteneur et exécuté une image légère.

Docker + Flask (API REST)

Dans le dossier 2decembre2, Dockerfile :

```
FROM python:3.12-slim  
WORKDIR /app  
RUN pip install flask  
COPY app.py .  
EXPOSE 5000  
CMD ["python", "app.py"]
```

L'application app.py expose plusieurs routes (/ , /message, /students en GET/POST/PUT) pour simuler une petite API REST de gestion d'étudiants.

Introduction à Kubernetes avec Minikube

Après installation de Minikube et kubectl, le cluster est démarré, puis vérifié :

```
minikube start  
kubectl get nodes
```

Résultat :
minikube Ready control-plane

Création d'un déploiement Nginx

```
kubectl create deployment hello-nginx --image=nginx  
kubectl get pods
```

Un pod Nginx apparaît en état Running.

Scaling

```
kubectl scale deployment hello-nginx --replicas=3  
kubectl get pods
```

3 pods hello-nginx-... sont maintenant en cours d'exécution.

Exposer le service en NodePort

```
kubectl expose deployment hello-nginx --type=NodePort --port 80  
minikube service hello-nginx --url
```

L'application Nginx est accessible depuis le navigateur.

Suppression et YAML

On a ensuite supprimé le service et le déploiement :

```
kubectl delete service hello-nginx  
kubectl delete deployment hello-nginx
```

Puis on a utilisé un fichier YAML nginx-deployment.yaml (dossier 2decembre3).

Application :

```
kubectl apply -f nginx-deployment.yaml
```

Kubernetes crée automatiquement le déploiement et les pods à partir de ce manifeste déclaratif.

Conclusion

Ce TP m'a permis de :

- approfondir Docker : images, conteneurs, export/import, volumes, réseaux, ports
- voir la différence entre données dans le conteneur et données persistantes dans les volumes
- lancer un serveur MySQL complet dans un conteneur et vérifier la persistance de la base
- comprendre les principaux modes réseaux Docker (bridge, host, none, réseau custom)
- écrire et utiliser plusieurs Dockerfile (Alpine, Java multi-stage, Flask)
- faire le lien avec Kubernetes : Minikube, pods, deployments, scaling, services, NodePort, YAML.